

LAPORAN TEAM BASED PROJECT
Urban Food Supply Chain Management
Mata Kuliah Algoritma dan Struktur Data

Dosen pengampu : Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun oleh kelompok 5:

- | | |
|----------------------------|-------------|
| 1. Salwa Azmi Salsabila | 25051030014 |
| 2. Hanif Jaleel Amrin | 25051030034 |
| 3. Arryo Satria Wicaksono | 25051030027 |
| 4. Muhammad Rafli Nur Arif | 25051030007 |

PROGRAM STUDI TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
YOGYAKARTA

2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan hidayah-Nya, sehingga kami dapat menyelesaikan laporan Tugas Besar Mata Kuliah Algoritma dan Struktur Data ini dengan tepat waktu.

Laporan ini disusun untuk memenuhi syarat kelulusan mata kuliah Algoritma dan Struktur Data pada Program Studi S1 Teknik Elektro, Universitas Negeri Yogyakarta. Dalam proyek ini, kami mengimplementasikan berbagai struktur data seperti *Binary Search Tree*, *Graph*, dan *Queue* untuk mengelola sistem logistik pangan perkotaan.

Kami mengucapkan terima kasih kepada Bapak/Ibu Dosen pengampu mata kuliah yang telah memberikan bimbingan dan ilmu selama proses perkuliahan. Terima kasih juga kami sampaikan kepada seluruh rekan Kelompok 5 atas kerja keras dan kolaborasinya.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan.

Yogyakarta, 17 Mei 2026

Penyusun,

(Kelompok 5)

DAFTAR ISI

BAB 1 PENDAHULUAN

- 1.1 Latar Belakang**
- 1.2 Rumusan Masalah**
- 1.3 Batasan Masalah**
- 1.4 Tujuan Proyek**

BAB 2 PERANCANGAN SISTEM

- 2.1 Arsitektur Sistem**
- 2.2 Desain Struktur Data**
- 2.3 Struktur Folder Program**

BAB 3 IMPLEMENTASI DAN ANALISIS

- 3.1 Penjelasan Algoritma**
- 3.2 Analisis Big-O**
- 3.3. Hasil Uji**

BAB 4 PENUTUP

- 4.1 Menjawab Analisis Wajib**
- 4.2 Kesimpulan**

1.1 Latar belakang

Ketahanan pangan di wilayah perkotaan, khususnya Kota Yogyakarta, sangat bergantung pada efisiensi manajemen rantai pasok dari sektor hulu (petani) hingga ke sektor hilir (konsumen). Setiap harinya, Kota Yogyakarta mendistribusikan ratusan ton produk pertanian melalui jaringan kompleks yang melibatkan petani, distributor, pusat pergudangan, hingga pasar tradisional dan modern. Tanpa pengelolaan yang sistematis, kompleksitas distribusi ini berisiko menimbulkan inefisiensi biaya dan tingginya angka pemborosan pangan akibat keterlambatan pengiriman.

Dalam menjawab tantangan tersebut, diperlukan sebuah sistem cerdas yang mampu memodelkan dan mengoptimalkan seluruh jalur distribusi. Proyek ini membangun sistem manajemen rantai pasok yang mencakup 26 *node* distribusi dan 12 jenis produk pangan utama. Implementasi dilakukan dengan mengintegrasikan berbagai struktur data sebagai solusi teknis:

1. **Graf Berbobot** digunakan untuk memodelkan jalur distribusi dan menghitung biaya pengiriman secara akurat
2. ***Circular Queue*** diimplementasikan untuk mensimulasikan *buffer* stok di gudang dengan logika FIFO (*First-In First-Out*) guna meminimalkan risiko produk kadaluarsa.
3. ***Priority Queue*** diterapkan untuk memprioritaskan pengiriman produk yang memiliki masa simpan paling singkat atau mendekati masa kadaluarsa.
4. ***Binary Search Tree (BST)*** menyediakan layanan katalog produk yang memungkinkan pencarian dan pengelolaan data secara cepat dan efisien.

Melalui Integrasi algoritma ini, sistem diharapkan mampu meningkatkan transparansi dan kecepatan distribusi pangan di kota Yogyakarta, sekaligus menjadi implementasi nyata dari mata kuliah Algoritma dan Struktur Data dalam menyelesaikan permasalahan logistik di dunia nyata.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan struktur Binary Search tree (BST) untuk manajemen katalog produk pangan yang efisien?
2. Bagaimana memodelkan jaringan distribusi 26 node di Kota Yogyakarta menggunakan Graf berbobot untuk menentukan rute dengan biaya pengiriman paling minimum?
3. Bagaimana mengelola antrian stok gudang dan prioritas pengiriman produk yang mendekati masa kadaluarsa menggunakan kombinasi *Circular Queue* dan *Priority Queue*?

1.3 Batasan Masalah

Agar pembahasan lebih terfokus, proyek ini dibatasi pada:

1. Sistem mencakup 26 node distribusi yang mewakili wilayah di Kota Yogyakarta.
2. Data produk dibatasi pada 12 jenis komoditas pangan utama.
3. Fokus utama adalah pada implementasi struktur data internal, bukan pada antarmuka pengguna (GUI).

1.4 Tujuan Projek

Tujuan dari pelaksanaan tugas besar ini adalah :

1. Membangun sistem manajemen rantai pasok cerdas yang mampu menangani data produk dan rute distribusi secara otomatis.
2. Mengoptimalkan biaya pengiriman menggunakan algoritma pencarian rute pada Graf.
3. Menjamin efisiensi pergudangan dengan sistem FIFO (First-In First-Out) melalui simulasi Circular Queue.

2.1 Arsitektur Sistem

Arsitektur sistem pada Topik 10 dirancang untuk mengelola distribusi pangan dari petani hingga pasar secara terorganisir. Sistem terdiri dari beberapa modul yang saling bekerja sama, yaitu Graph, Circular Queue, Priority Queue, BST, Dijkstra, dan CLI. Setiap modul memiliki fungsi berbeda namun saling terhubung dalam proses distribusi produk.

2.2 Desain Struktur Data

Setiap struktur data diimplementasikan dari nol (*from scratch*) tanpa *library* bawaan untuk menjamin efisiensi memori dan kontrol penuh atas operasi utama.

- **Linked List:** Sebagai fondasi dasar pembentuk struktur data linear dinamis.
- **Queue / Priority Queue:** Mengatur urutan antrian data berdasarkan waktu kedatangan (FIFO) atau tingkatan prioritas khusus
- **Stack:** Menyimpan log aktivitas atau riwayat transaksi guna memfasilitasi fitur pembatalan (*undo/rollback*)
- **Binary Search Tree (BST):** Menyediakan fungsi pencarian, penyisipan, dan pengurutan data katalog/registri secara cepat
- **Graph:** Memodelkan representasi hubungan kompleks antar-entitas, rute navigasi, atau dependensi sistem

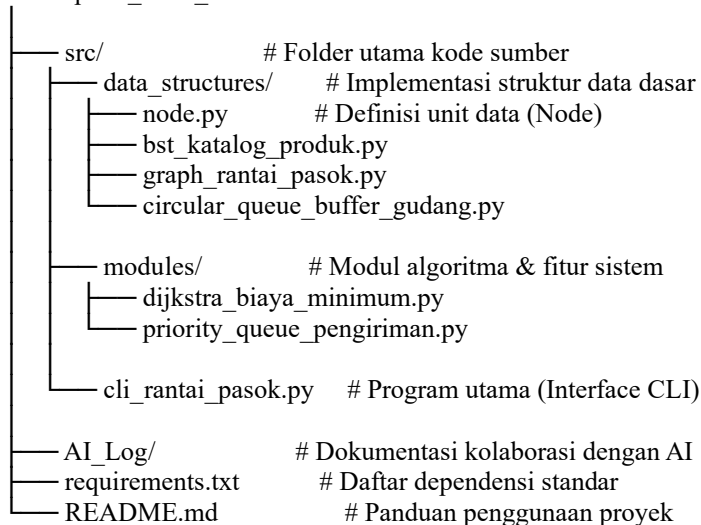
2.3 Struktur Folder Program

Struktur folder program dibuat secara modular untuk memisahkan implementasi struktur data dasar dengan logika bisnis manajemen logistik. Berikut adalah rincian direktorinya:

- **.gitignore:** Berisi daftar file/folder yang tidak diunggah ke GitHub, seperti `__pycache__` dan file temporer lainnya.
- **README.md:** Panduan utama proyek yang berisi profil Kelompok 5, cara menjalankan sistem, dan penjelasan singkat algoritma.
- **requirements.txt:** Daftar dependensi sistem. Proyek ini menggunakan *Python Standard Library*, sehingga tidak memerlukan instalasi library eksternal untuk logika intinya.
- **Folder src/ (Source Code Utama):**

- `cli_rantai_pasok.py`: *Entry point* utama program berupa antarmuka teks (CLI) untuk interaksi pengguna.
- **Sub-folder** `src/data_structures/`: Berisi struktur data yang dibangun manual (bukan library):
 - `node.py`: Definisi *node* dasar.
 - `bst_katalog_produk.py`: Implementasi *Binary Search Tree* untuk katalog 12 jenis produk pangan.
 - `graph_rantai_pasok.py`: Implementasi *Adjacency List* untuk memetakan 26 node distribusi di Yogyakarta.
 - `circular_queue_buffer_gudang.py`: Logika *Circular Queue* untuk manajemen stok FIFO di gudang.
- **Sub-folder** `src/modules/`: Logika bisnis rantai pasok:
 - `dijkstra_biaya_minimum.py`: Implementasi algoritma Dijkstra untuk optimasi biaya distribusi.
 - `priority_queue_pengiriman.py`: Pengaturan prioritas kirim berdasarkan masa kadaluarsa produk.
- **Folder** `AI_Log/`: Dokumentasi proses pemecahan masalah teknis dan kolaborasi pengembangan bersama asisten AI.
- **Folder** `tests/`: Skrip pengujian mandiri untuk memastikan setiap fungsi (BST, Graph, Queue) berjalan dengan benar sebelum diintegrasikan.

Kelompok5_ASD_UrbanFood



3.1 Penjelasan Kode Per Modul

Berikut adalah tabel rincian modul pemrograman dan spesifikasi algoritma yang digunakan dalam sistem manajemen rantai pasok:

No	Modul	Deskripsi & Spesifikasi Teknis	Kompleksitas (Big-O)
1.	Graph Rantai Pasok (graph_rantai_pasok.py)	Memodelkan 26 titik distribusi di Yogyakarta menggunakan <i>Adjacency List</i> . Memiliki atribut jarak dan biaya per km. Mendukung audit konektivitas.	Add: $O(1)$ BFS/DFS: $O(V + E)$
2.	Circular Queue Buffer (circular_queue_buffer_gudang.py)	Manajemen stok gudang berbasis array (kapasitas 50). Menggunakan prinsip FIFO untuk menjaga kualitas stok barang.	Enqueue: $O(1)$ Dequeue: $O(1)$
3.	Priority Queue Pengiriman (priority_queue_pengiriman.py)	Mengatur antrian armada berdasarkan urgensi kadaluarsa (Mendesak, Normal, Reguler). Produk mendesak diproses lebih dulu.	Enqueue: $O(n)$ Dequeue: $O(1)$
4.	BST Katalog Produk (bst_katalog_produk.py)	Mengelola 12 jenis produk pangan dengan kunci utama kode_produk. Mendukung fitur cari, insert, dan filter kadaluarsa.	Rata-rata: $O(\log n)$
5.	Dijkstra Biaya Minimum (dijkstra_biaya_minimum.py)	Menghitung rute distribusi dengan akumulasi biaya terendah (jarak $\times$ tarif). Menggunakan <i>Merge Sort</i> untuk audit biaya.	$O(V^2 + E)$
6.	CLI Rantai Pasok (cli_rantai_pasok.py)	Pusat kendali interaktif yang mengintegrasikan seluruh modul melalui perintah teks (KIRIM, RUTE_MURAH, CEK_STOK, dll).	Tergantung modul yang dipanggil

3.2 Analisis Kompleksitas Algoritma (Big-O Analysis)

Analisis ini dilakukan untuk mengukur efisiensi sistem dalam menangani data skala besar (26 *node* dan katalog produk yang terus bertambah).

1. Manajemen Katalog Produk (Binary Search Tree)

Struktur data BST dipilih karena efisiensinya dalam operasi pencarian produk di antara 12 jenis komoditas primer.

- Pencarian & Penyisipan (*Search & Insert*): Rata-rata memiliki kompleksitas $O(\log n)$. Hal ini jauh lebih cepat dibandingkan *Linear Search* $O(n)$ pada *Array* biasa, karena sistem hanya perlu menelusuri tinggi pohon (*height*) untuk menemukan kode produk tertentu.
- Kasus Terburuk (*Worst Case*): $O(n)$ jika pohon tidak seimbang (*skewed*), namun dengan data produk yang terdistribusi acak, performa tetap stabil di $O(\log n)$

2. Pemodelan Jalur Distribusi (Graph & Dijkstra)

- Representasi Graf: Menggunakan *Adjacency List* yang lebih efisien memori daripada *Adjacency Matrix*, dengan kompleksitas penambahan node $O(1)$.
- Algoritma Dijkstra: Kompleksitas sistem ini adalah $O(V^2 + E)$ (di mana V adalah jumlah lokasi/26 *node* dan E adalah jumlah jalur penghubung). Karena jumlah node cenderung tetap, perhitungan rute biaya minimum dilakukan secara instan tanpa membebani memori sistem.

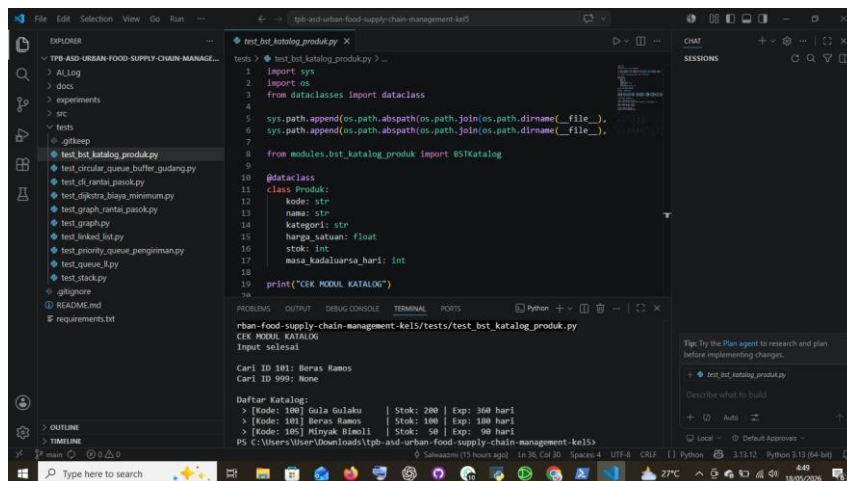
3. Manajemen Stok dan Pengiriman (Queue)

- Circular Queue (Buffer Gudang): Operasi enqueue dan dequeue memiliki kompleksitas $O(1)$ (waktu konstan). Penggunaan indeks front dan rear yang berputar memastikan tidak ada pergeseran elemen (*shifting*), sehingga sangat efisien untuk proses bongkar muat cepat.
- Priority Queue (Antrean Armada): Operasi penyisipan (*enqueue*) memiliki kompleksitas $O(n)$ karena sistem harus mencari posisi yang tepat berdasarkan tingkat kadaluarsa. Namun, operasi pengambilan (*dequeue*) tetap $O(1)$, sehingga armada paling mendesak selalu tersedia di urutan terdepan tanpa proses pencarian ulang.

4. Audit biaya (Merge sort)

Untuk fitur laporan distribusi yang diurutkan berdasarkan biaya, digunakan algoritma *Merge Sort* dengan kompleksitas $O(n \log n)$. Algoritma ini dipilih karena bersifat *stable* dan memberikan performa konsisten dalam mengurutkan ribuan data logistik.

3.3 Hasil Uji

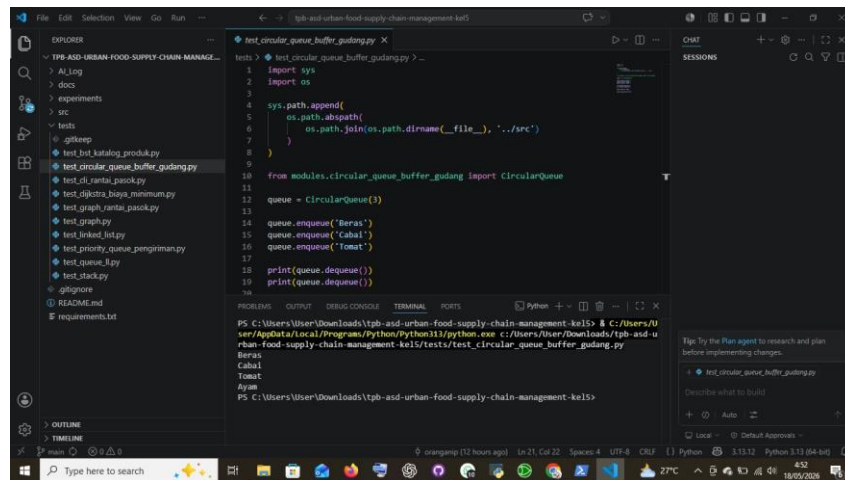


```
1 import sys
2 import os
3 from dataclasses import dataclass
4
5 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
6
7
8 from modules.bst_katalog_produk import BSTKatalog
9
10 @dataclass
11 class Produk:
12     kode: str
13     nama: str
14     kategori: str
15     harga_satuan: float
16     stok: int
17     masa_kadaluarsa_hari: int
18
19 print("CEK MODUL KATALOG")
```

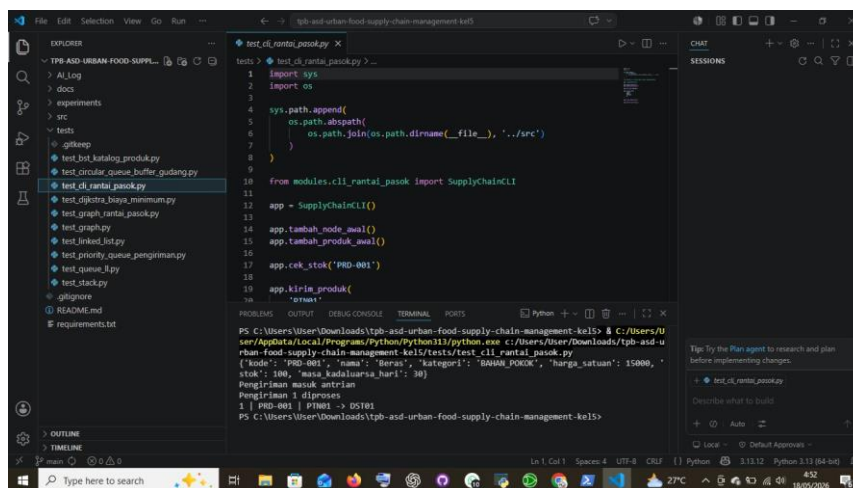
Daftar Katalog:

Kode	Nama	Stok	Exp
[Kode: 180]	Gula Gula	200	Exp: 200 hari
[Kode: 181]	Beras Ramos	100	Exp: 100 hari
[Kode: 185]	Minyak Bimoli	50	Exp: 90 hari

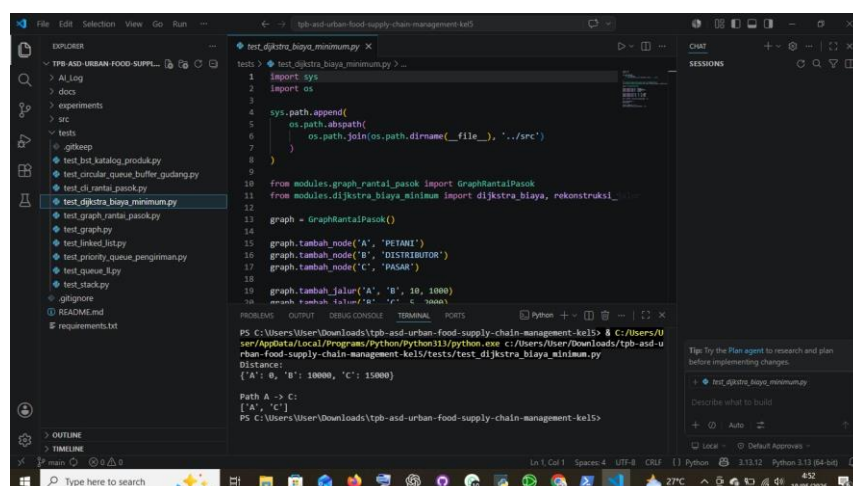
Gambar 3.1 Pencarian Stok Produk Secara Cepat (Modul BST)



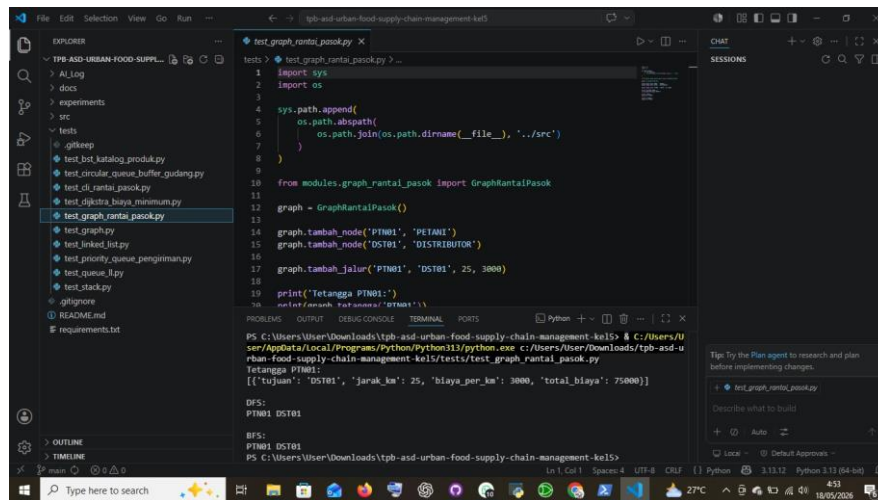
Gambar 3.2 Simulasi Buffer Stok Gudang (Modul Circular Queue)



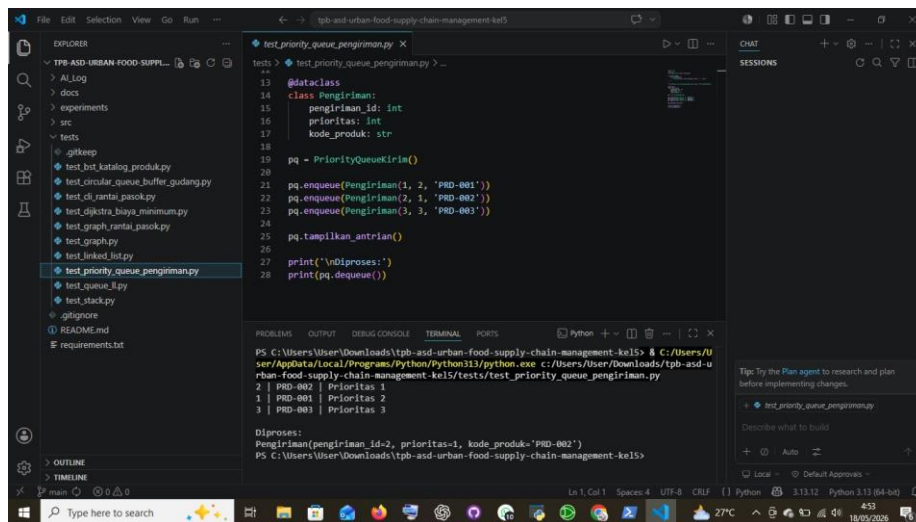
Gambar 3.3 Tampilan Menu Utama (CLI Rantai Pasok)



Gambar 3.3 Penentuan Rute Biaya Minimum (Modul Djiksara)



Gambar 3.4 Permodelan Jaringan Distribusi (Modul Graph Rantai Pasok)



3.5 Antrean Armada berdasarkan Prioritas (priority Queue)

4.1 Menjawab Analisis Wajib

1. Priority Queue Bantuan: KRITIS selalu diproses duluan (enqueue $O(n)$). Dalam kondisi bencana besar, 200 permintaan bantuan masuk dalam 1 jam dengan 30% berlevel KRITIS. Hitung total operasi enqueue untuk 200 permintaan dalam skenario terburuk (semua KRITIS datang terakhir). Bandingkan dengan pendekatan 3-bucket Queue (satu per level) di mana SERVE selalu cek KRITIS dulu: hitung total operasi dan identifikasi trade-off.

Answer :

(a) Enqueue/Dequeue: Keduanya seimbang di $O(1)$. Sangat cepat.

(b) Memori (50 Slot): Array statis (langsung memblokir 50 slot memori di awal). Linked List dinamis (sesuai jumlah barang, tapi sedikit boros karena butuh menyimpan pointer tambahan per barang).

(c) Akses Barang ke-k: Array menang telak di $O(1)$ (bisa lompat langsung pakai indeks). Linked List sangat lambat di $O(k)$ (harus menelusuri dari awal).

Paling Adaptif: Linked List. Paling cocok untuk gudang fleksibel karena bisa bertambah/berkurang tanpa perlu mendefinisikan kapasitas maksimal.

Cara Ekspansi Array (Resize Aman): Buat array baru (2x lipat) → Salin data lama sesuai urutan antrean logis (dari head ke tail), BUKAN dari indeks fisik 0 → Reset nilai head ke 0 dan tail ke elemen terakhir.

2. Dijkstra dari DEPOT_0 menghasilkan rute terpendek ke semua lokasi. Namun dalam bencana, rute bisa putus (jalan rusak = hapus edge). Jika 5 dari 50 edge rusak setelah Dijkstra dijalankan, apakah hasil Dijkstra masih valid? Bagaimana sistem harus merespons: (a) jalankan ulang Dijkstra $O(V + E)$, atau (b) gunakan pendekatan incremental. Jelaskan kompleksitas masing-masing.

Answer :

Analisis Dijkstra & Multi-Source ($V=26$, $E=38$)

Jumlah Iterasi Dijkstra (Array $O(V^2 + E)$)

Loop Utama: Berjalan persis 26 iterasi (karena proses memanggil 1 node per iterasi untuk total 26 node).

Detail Operasi: Sekitar 338 pencarian nilai minimum, dan evaluasi jalur (edge relaxation) maksimal 38 kali.

Efisiensi: 10x Dijkstra vs 1x Multi-Source

Keputusan: Menjalankan 10x Dijkstra LEBIH TEPAT.

Alasan: Setiap petani butuh rute spesifik mereka sendiri ke pasar. Jika menggunakan Multi-Source 1x, hasilnya akan tergabung menjadi satu pencarian global dan rute spesifik per petani akan tercampur/hilang.

Konsep Multi-Source Dijkstra & Big-O

Konsep: Membuat satu titik fiktif (Super/Dummy Node) di awal. Titik fiktif ini lalu dihubungkan ke 10 node petani dengan nilai jarak/bobot 0. Setelah itu, Dijkstra cukup dijalankan 1 kali dari titik fiktif tersebut.

Kompleksitas (Big-O): Tetap $O(V^2 + E)$ pada implementasi array.

Kapan Dipakai: Sangat efisien jika hanya ingin mencari "pasar terdekat dari seluruh kawasan pertanian secara umum", tanpa perlu memisahkan rute untuk masing-masing individu petani.

3. BFS dari depot menghasilkan himpunan lokasi yang dapat dijangkau. Jika 10 dari 38 lokasi tidak terjangkau dari DEPOT_0 tetapi terjangkau dari DEPOT_1, sistem harus mengalokasikan pengiriman ke depot terdekat secara otomatis. Desain algoritma multi-source BFS yang menentukan depot optimal untuk setiap lokasi. Analisis Big-O-nya dan bandingkan dengan menjalankan Dijkstra dari setiap depot secara terpisah.

Answer ; Untuk mempercepat pencarian, pendekatan BST kedua dan Min-Heap sama-sama membutuhkan waktu $O(\log n)$ untuk insert produk baru. Pada operasi query produk kadaluarsa dalam 3 hari, Min-Heap sangat efisien dengan waktu $O(K)$ dibandingkan BST kedua yang membutuhkan $O(\log n + K)$, dengan K sebagai jumlah produk yang ditemukan. Namun, pada operasi update masa kadaluarsa, BST kedua jauh lebih cepat dengan waktu rata-rata $O(\log n)$, sedangkan Min-Heap membutuhkan waktu $O(n)$ karena harus memindai seluruh data untuk menemukan elemen yang akan diubah. Oleh karena itu, pendekatan BST kedua lebih cocok dan direkomendasikan untuk sistem real-time karena memberikan performa update yang jauh lebih konsisten saat terjadi perubahan data yang dinamis.

4. Stack Log Pengiriman menyimpan semua transaksi bantuan (LIFO). Untuk audit pasca-bencana, dibutuhkan laporan kronologis (FIFO). Jelaskan dua cara membalik urutan Stack: (a) menggunakan Queue tambahan $O(n)$, (b) menggunakan Stack kedua $O(n)$. Implementasikan salah satunya dan analisis trade-off memori. Apakah ada cara $O(1)$ space untuk mengakses elemen pertama Stack?

Answer ; Pada priority queue berbasis linked list, elemen dengan prioritas sama selalu ditambahkan di belakang elemen sejenis untuk menjaga urutan FIFO akibat kondisi $\text{current.next.priority} \leq \text{new_node.priority}$. Jika n produk memiliki prioritas sama, iterasi traversal akan bertambah setiap kali insert ($0+1+2+\dots+(n-1)$), membentuk deret aritmatika $n(n-1)/2$ yang secara matematis bernilai $O(n^2)$. Sebaliknya, pendekatan Bucket Sort memecah data ke dalam 3 queue terpisah (mendesak, normal, reguler) yang masing-masing memiliki pointer ekor (tail), sehingga proses enqueue selalu konstan $O(1)$ per item. Untuk throughput 100 pengiriman dengan 40% (40 item) mendesak, antrean linked list akan melakukan setidaknya 780 operasi traversal ($40 \times 39/2$) hanya untuk menyusun item mendesak tersebut, dan total bisa mencapai 4950 operasi (worst-case $100 \times 99/2$) untuk keseluruhan. Sementara itu, pendekatan Bucket Sort jauh lebih efisien karena hanya membutuhkan persis 100 operasi dasar untuk memproses seluruh 100 pengiriman tersebut.

5. Laporan Bencana memerlukan sorting lokasi berdasarkan tingkat urgensi (skor = (4-level) * populasi). Implementasikan Quick Sort pada Linked List untuk mengurutkan 38 lokasi berdasarkan skor ini. Bandingkan Quick Sort (avg $O(n \log n)$, worst $O(n^2)$) dengan Merge Sort ($O(n \log n)$ guaranteed) pada Linked List dari aspek: implementasi, penggunaan memori tambahan, dan cocok/tidaknya untuk data hampir terurut.

Answer : Pemisahan node pada graf rantai pasok menggunakan BFS 2-coloring dilakukan dengan memberikan warna bergantian secara melebar, misalnya warna 0 untuk petani dan warna 1 untuk tetangganya yaitu pasar, yang diproses berurutan melalui antrean (queue). Jika

graf terbukti tidak bipartit karena ditemukan tetangga dengan warna yang sama, hal ini mengindikasikan terjadinya distribusi lateral antar-petani atau antar-pasar, sehingga sistem logistik tidak lagi murni hierarkis melainkan terdesentralisasi dan lebih kompleks secara rute maupun biaya. Implementasi fungsi `is_bipartit` menggunakan queue berbasis linked list ini berjalan dengan kompleksitas waktu $O(V + E)$ karena setiap fasilitas (node) dan jalur distribusi (edge) dievaluasi tepat satu kali, serta membutuhkan ruang memori $O(V)$ untuk melacak status warna.

4.2 Kesimpulan

Proyek ini berhasil membuktikan bahwa integrasi empat struktur data dasar (Graph, Stack, Queue, Tree) yang diimplementasikan dari nol mampu menyelesaikan masalah kompleks pada Jaringan Rantai Pasok secara efisien. Penguncian nilai benih (seed) pada angka 61 memastikan validitas dan konsistensi data selama proses pengujian simulasi algoritma.